

THE COMPLETE GUIDE TO

Data Visualization in Python

Matplotlib · Seaborn · Plotly · NumPy

Every Plot Type · Python Code · ML/DL/CV Use Cases · Edge Cases · Diagrams

For NAVTTC Data Science Program · 2026

Table of Contents

- [1. Matplotlib — The Foundation](#)
- [2. Line Plot](#)
- [3. Scatter Plot](#)
- [4. Bar Plot](#)
- [5. Histogram & KDE](#)
- [6. Box Plot](#)
- [7. Violin Plot](#)
- [8. Distribution Plots \(Seaborn\)](#)
- [9. Categorical Plots \(Seaborn\)](#)
- [10. Matrix Plots / Heatmaps](#)
- [11. Regression Plots](#)
- [12. Pie Chart & Donut Chart](#)
- [13. Area Chart](#)
- [14. Hexbin Plot](#)
- [15. ROC Curve & AUC](#)
- [16. Feature Importance Plot](#)
- [17. t-SNE / UMAP Visualization](#)
- [18. Plotly — Interactive Plots](#)
- [19. Working with Images \(CV\)](#)
- [20. Axis, Titles & Professional Styling](#)
- [21. Quick Reference — Choose the Right Plot](#)

1. Matplotlib — The Foundation

Matplotlib is Python's most popular plotting library. It was created by John D. Hunter in 2003 and is now the backbone of almost every other visualization tool in Python (Seaborn, Pandas plotting, etc.).

Think of Matplotlib as a blank canvas. You decide every single thing: the colors, the sizes, the labels, the layout. This makes it incredibly flexible — but also verbose.

1.1 Two Ways to Plot

The Functional Way (plt.plot)

Quick and simple. Works like MATLAB. Great for one-off charts.

Functional / pyplot Style

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = x**2

plt.plot(x, y)
plt.title("Hello World in Matplotlib")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.show()
```

The Object-Oriented Way (fig, ax)

Professional way. Gives you full control. Use this in real projects and when making subplots.

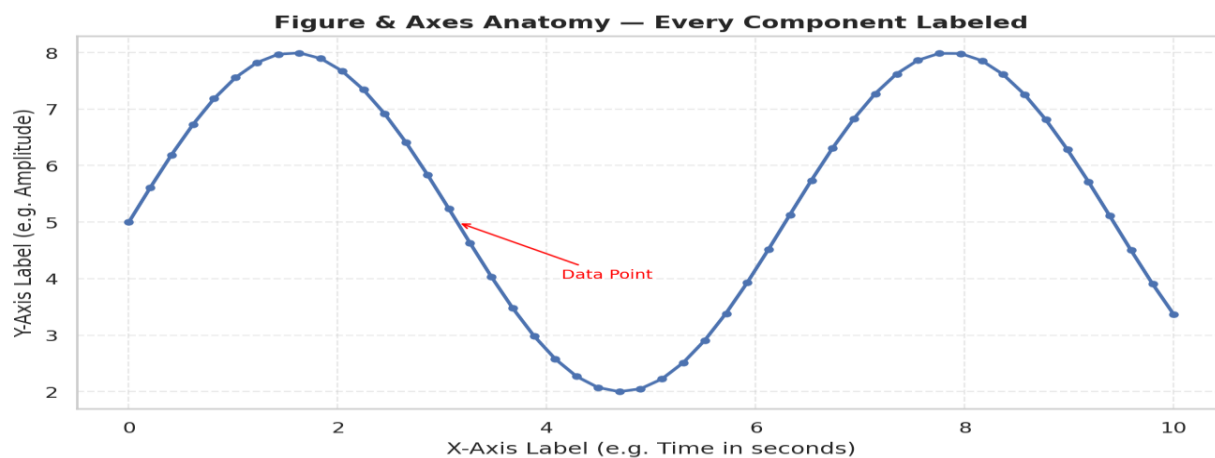
Object-Oriented Style

```
fig, ax = plt.subplots(figsize=(10, 6)) # Create figure + axes

ax.plot(x, y, color='blue', linewidth=2, label='y = x²')
ax.set_title('OO Way Plot', fontsize=14, fontweight='bold')
ax.set_xlabel('X Variable')
ax.set_ylabel('Y Variable')
ax.legend()
plt.tight_layout()
plt.show()
```

1.2 Figure Anatomy

Every Matplotlib plot is made of several nested components. Understanding this helps you customize any part of a plot.

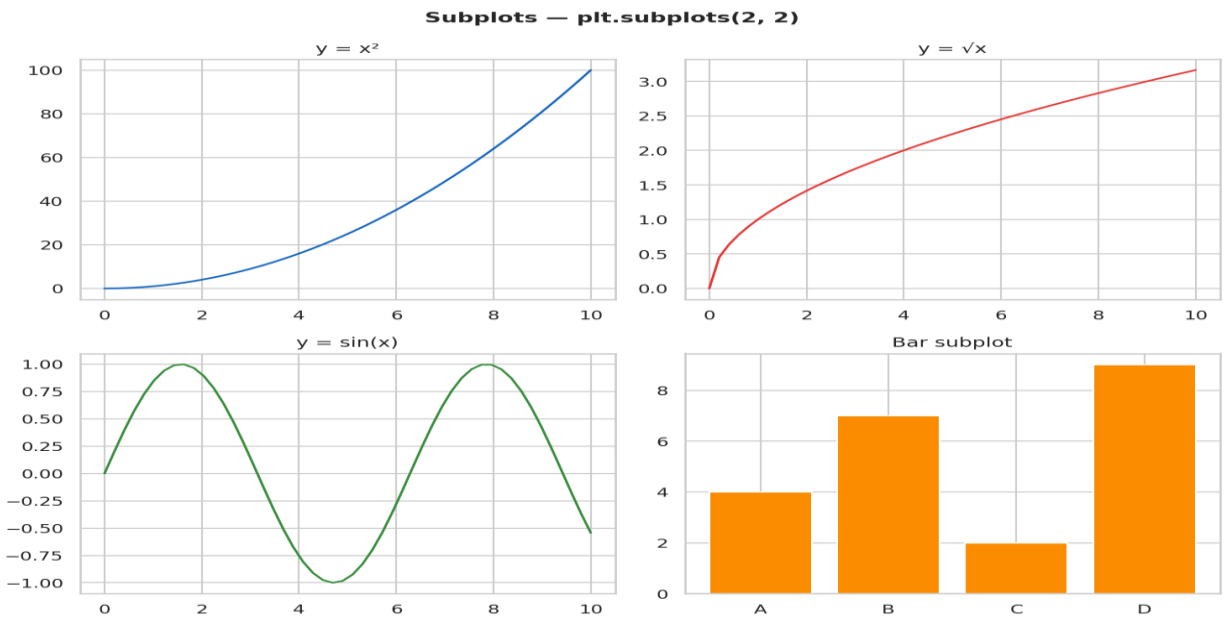


i Figure = the entire window/image. Axes = the actual plot area inside it. You can have multiple Axes in one Figure.

Component	What It Is	How to Control
Figure	The whole canvas/image	<code>plt.figure(figsize=(10,6))</code>
Axes	The plot area (one chart)	<code>fig, ax = plt.subplots()</code>
X-Axis / Y-Axis	The measurement scales	<code>ax.set_xlabel()</code> , <code>ax.set_ylabel()</code>
Title	Chart headline	<code>ax.set_title()</code>
Spine	The border lines around axes	<code>ax.spines['top'].set_visible(False)</code>
Ticks	Marks on the axis	<code>ax.tick_params()</code>
Grid	Background reference lines	<code>ax.grid(True, alpha=0.4)</code>
Legend	Key for multiple series	<code>ax.legend()</code>

1.3 Subplots

Subplots let you put multiple charts side by side in one figure. You create a grid of axes using `plt.subplots(rows, cols)`.

**Creating a 2x2 Grid of Subplots**

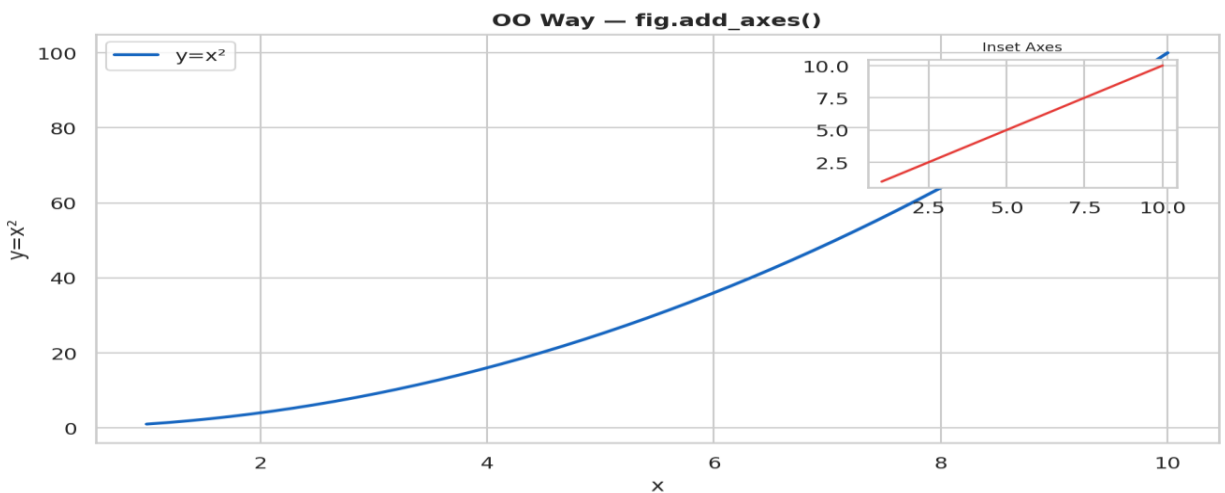
```
fig, axes = plt.subplots(2, 2, figsize=(12, 8)) # 2 rows, 2 cols
fig.suptitle('My Dashboard', fontsize=16) # overall title

axes[0, 0].plot(x, y); axes[0, 0].set_title('Top Left')
axes[0, 1].scatter(x, y); axes[0, 1].set_title('Top Right')
axes[1, 0].bar(cats, vals); axes[1, 0].set_title('Bottom Left')
axes[1, 1].hist(data); axes[1, 1].set_title('Bottom Right')

plt.tight_layout() # prevents overlap
plt.show()
```

1.4 Object-Oriented: fig.add_axes()

For more control, you can manually place axes anywhere on the figure using pixel-like fractions: [left, bottom, width, height] — all from 0 to 1.

**Manual Axis Placement (Inset Plots)**

```
fig = plt.figure(figsize=(10, 6))

# Main plot: starts 10% from left, 10% from bottom, 80% wide, 80% tall
```

```
ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax1.plot(x, y, label='Main')

# Inset/small plot inside the big one
ax2 = fig.add_axes([0.65, 0.65, 0.22, 0.22])
ax2.plot(x, np.sqrt(y), color='red')
ax2.set_title('Inset', fontsize=8)

plt.show()
```

1.5 Styling Lines and Points

Line Styles, Colors, Markers

```
# Colors: named ('red'), hex ('#FF5722'), RGB ((0.2, 0.4, 0.8))
ax.plot(x, y, color='steelblue', linewidth=2.0, linestyle='-', label='Solid')
ax.plot(x, z, color='#E53935', linewidth=1.5, linestyle='--', label='Dashed')
ax.plot(x, a, color='green', linewidth=1.0, linestyle='-.', label='Dash-Dot')
ax.plot(x, b, color='purple', linewidth=0.8, linestyle=':', label='Dotted')

# Markers: 'o'=circle, 's'=square, '^'=triangle, 'D'=diamond, '+'=plus
ax.plot(x, y, marker='o', markersize=5, markerfacecolor='white')

# Combined shorthand: color + marker + line
ax.plot(x, y, 'b-o') # blue solid line with circle markers
```

⚠ EDGE CASE — Overlapping lines: If lines overlap, use `alpha=0.6` to make them semi-transparent. Or offset them slightly.

1.6 Global Styling with rcParams

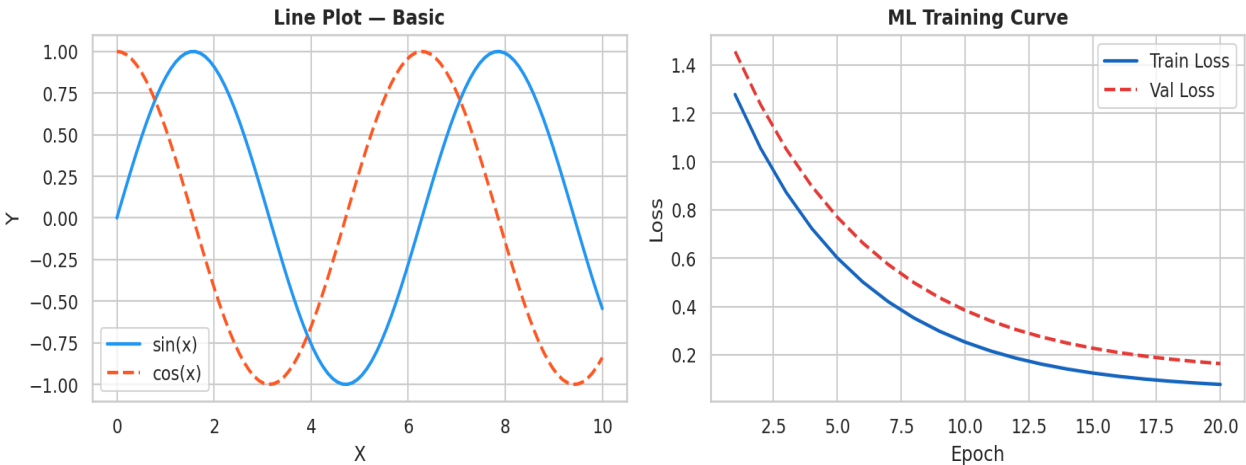
Set Consistent Style for All Plots

```
plt.rcParams.update({
    'font.family': 'Arial',
    'font.size': 11,
    'axes.titlesize': 14,
    'axes.titleweight': 'bold',
    'axes.labelsize': 12,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'grid.linestyle': '--',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 150,
})
# Now every plot you make will use these defaults
```

2. Line Plot

A line plot connects data points with a continuous line. It is the go-to chart whenever you have **ordered data** — most commonly data that changes over time.

Think of it like a heart rate monitor: time on X, value on Y, and the line shows how the value evolves.



2.1 When to Use Line Plots

Use Case	Example
Time series data	Stock prices, temperature over weeks, website traffic by day
ML/DL training curves	Plot loss and accuracy per epoch to detect overfitting
Mathematical functions	Visualizing $\sin(x)$, polynomials, activation functions
Continuous trends	Population growth, economic indicators over years
Comparing multiple sequences	Multiple models' performance vs. time

2.2 Python Code

Basic Line Plot

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(x, y, color='#2196F3', linewidth=2, label='sin(x)')
ax.set_title('Sine Wave', fontweight='bold')
ax.set_xlabel('X'); ax.set_ylabel('Y')
ax.legend()
plt.tight_layout()
plt.show()
```

ML Training Curve — Most Common DS Use Case

```
epochs = range(1, 51)
train_loss = [...] # your losses list
val_loss = [...]
```

```
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(epochs, train_loss, label='Train Loss', color='#1565C0', lw=2)
ax.plot(epochs, val_loss, label='Val Loss', color='#E53935', lw=2, ls='--')

# Vertical line to mark where overfitting starts
ax.axvline(x=30, color='orange', ls=':', lw=1.5, label='Overfit Start')

ax.set_xlabel('Epoch'); ax.set_ylabel('Loss')
ax.set_title('Training vs Validation Loss', fontweight='bold')
ax.legend()
plt.tight_layout(); plt.show()
```

2.3 Edge Cases & Pitfalls

Problem	Cause	Fix
Flat/no line visible	Data not sorted by X	Sort data first: <code>df.sort_values('x')</code>
Jagged zigzag line	X values out of order	Use <code>np.sort(x)</code> before plotting
Line disappears at edge	X range too narrow	Use <code>ax.set_xlim(min, max)</code>
Y-axis doesn't start at 0	Auto-scaling	Use <code>ax.set_ylim(0, max_val)</code> for bar-like data
Too many lines = messy	Overplotting	Use <code>alpha=0.4</code> or highlight top 3, gray rest
Missing data gaps	NaN values in series	Plot will break at NaN — forward-fill or mark gaps

2.4 DS / ML / DL Use Cases

✓ML: Always plot both train and val loss together. A growing gap = overfitting. Both decreasing = healthy.

ML: Training/validation loss curves, learning rate schedules, accuracy per epoch

DL: Per-layer gradient norms over training (detect vanishing/exploding gradients)

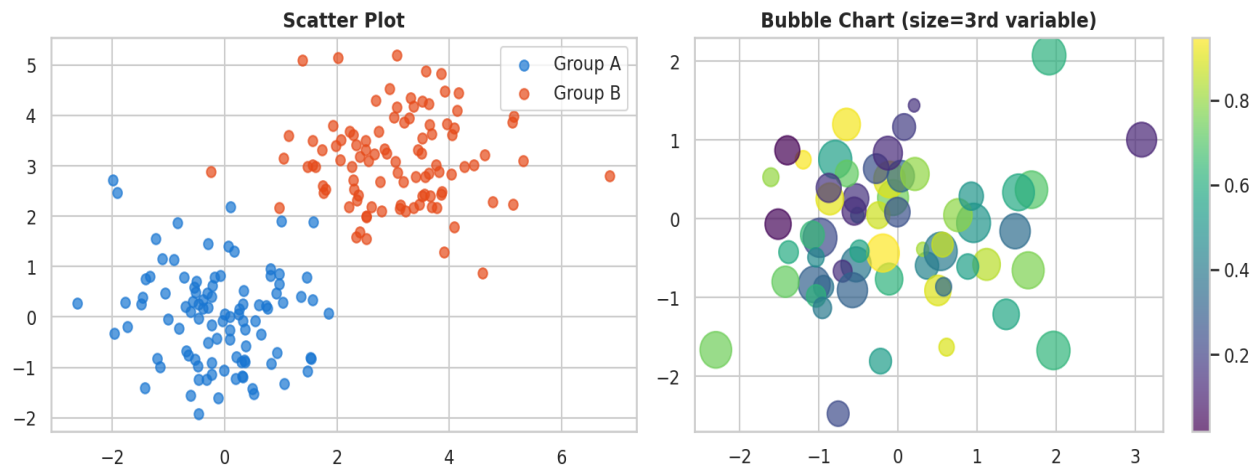
CV: mAP (mean average precision) per epoch for object detection models

Data Science: Time-series analysis, sales trends, anomaly detection baselines

3. Scatter Plot

A scatter plot places individual data points as dots on a 2D coordinate system. Each point has an X value and a Y value. It is used to **show the relationship (correlation) between two numeric variables**.

Imagine two exam scores for 100 students: plot math score on X and science score on Y. If students who score high in math also score high in science, the dots form a diagonal line — that's positive correlation.



3.1 When to Use Scatter Plots

Use Case	Example
Correlation analysis	Does income correlate with education level?
Outlier detection	Spot data points far from the cluster
Cluster visualization	Show groups after K-Means or DBSCAN
Feature relationships	height vs weight, bill vs tip
t-SNE / UMAP projections	Visualize 512-dim embeddings in 2D

3.2 Python Code

```
Basic Scatter Plot
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
x = np.random.randn(200)
y = x * 1.5 + np.random.randn(200) * 0.5 # correlated with x

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(x, y, alpha=0.6, color='#1976D2', edgecolors='white', linewidths=0.5)
ax.set_xlabel('Feature X'); ax.set_ylabel('Feature Y')
ax.set_title('Scatter Plot - Positive Correlation', fontweight='bold')
plt.tight_layout(); plt.show()
```

Multi-Class Scatter (for ML clustering)

```
# Different colors per class
classes = [0, 0, 1, 1, 2, 2, ...] # labels
colors  = ['#1976D2', '#E53935', '#388E3C']

for cls in range(3):
    mask = (np.array(classes) == cls)
    ax.scatter(x[mask], y[mask], label=f'Class {cls}',
               alpha=0.7, s=50, color=colors[cls])
ax.legend(title='Class')
```

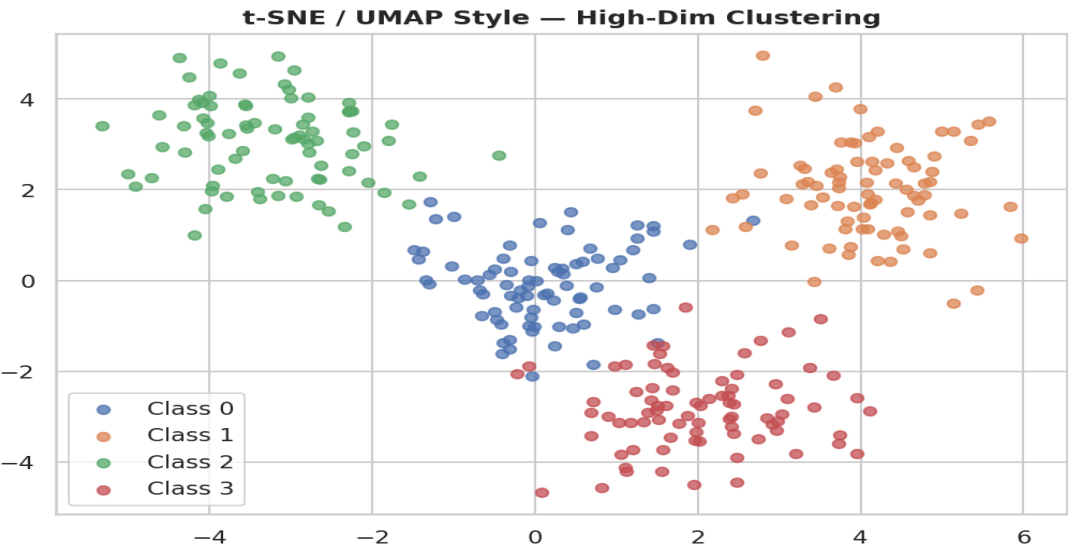
Bubble Chart — 3rd Variable as Size

```
# Size of dots encodes a 3rd variable
sizes = df['population'] / 1000 # scale to reasonable range
scatter = ax.scatter(df['gdp'], df['happiness'],
                    s=sizes, # size = population
                    c=df['continent_code'], # color = region
                    cmap='tab10', alpha=0.7)
plt.colorbar(scatter, label='Region')
```

3.3 Edge Cases & Pitfalls

Problem	Cause	Fix
All dots overlap (overplotting)	Too many points in same area	Use alpha=0.1, or hexbin, or jitter
Dots too small to see	Default size too small	Increase s=50 or s=100
Misleading bubble sizes	Encoding value directly as radius	Area should encode value: s = value * scale
Correlation looks wrong	Different scales on axes	Use ax.set_aspect('equal') or standardize data
Too many classes	> 8 different colors	Limit to top classes + "Other" group

3.4 DS / ML / DL / CV Use Cases



t-SNE / UMAP: After reducing high-dimensional data (e.g., 768-dim BERT embeddings) to 2D, scatter plots reveal natural clusters and class separability.

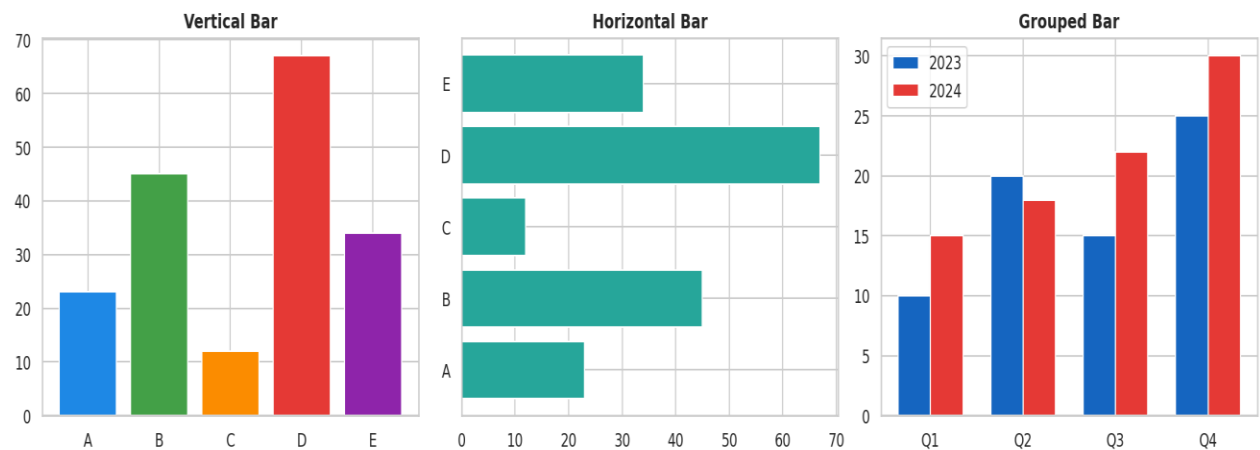
CV: Visualize image embeddings — each dot = one image. Dots of same class should cluster together (measures representation quality).

ML: Predicted vs Actual scatter — perfect model = diagonal line. Deviations reveal systematic errors.

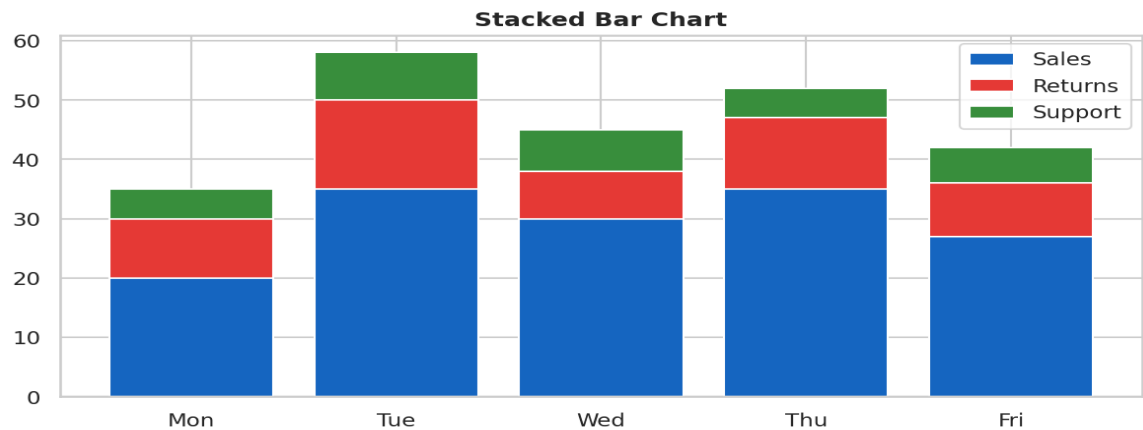
✓TIP: For datasets > 5000 points, switch to hexbin plot — it solves the overplotting problem and shows density much better.

4. Bar Plot

Bar plots use rectangular bars to compare values across categories. The height (or length) of each bar is proportional to the value it represents. It is one of the most universally understood chart types.



4.1 Types of Bar Plots



Type	When to Use	Code
Vertical Bar	Short category names, ≤ 10 categories	<code>ax.bar(x, y)</code>
Horizontal Bar	Long category names, many categories	<code>ax.barh(x, y)</code>
Grouped Bar	Compare sub-groups within each category	<code>ax.bar(x-w/2, y1, w); ax.bar(x+w/2, y2, w)</code>
Stacked Bar	Show part-to-whole across categories	<code>ax.bar(x, y1); ax.bar(x, y2, bottom=y1)</code>
100% Stacked Bar	Compare proportions (not absolute values)	Normalize each group to 100% first
Seaborn barplot	Auto-computes mean + confidence interval	<code>sns.barplot(x=..., y=..., data=...)</code>

4.2 Python Code

Vertical and Horizontal Bar

```
import matplotlib.pyplot as plt
import seaborn as sns

# Vertical bar
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(['A','B','C','D'], [23, 45, 12, 67],
       color=['#1E88E5', '#43A047', '#FB8C00', '#E53935'],
       edgecolor='white', linewidth=0.8)
ax.set_title('Category Comparison', fontweight='bold')
ax.set_ylabel('Value')

# Add value labels on top of bars
for bar in ax.patches:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
            f'{bar.get_height():.0f}', ha='center', va='bottom', fontsize=10)
```

Grouped Bar Chart

```
import numpy as np

categories = ['Q1', 'Q2', 'Q3', 'Q4']
vals_2023 = [10, 20, 15, 25]
vals_2024 = [15, 18, 22, 30]
x = np.arange(len(categories)); w = 0.35

fig, ax = plt.subplots(figsize=(9, 5))
bars1 = ax.bar(x - w/2, vals_2023, w, label='2023', color='#1565C0')
bars2 = ax.bar(x + w/2, vals_2024, w, label='2024', color='#E53935')
ax.set_xticks(x); ax.set_xticklabels(categories)
ax.legend(); ax.set_title('Revenue by Quarter', fontweight='bold')
```

Seaborn barplot — Auto Mean + Confidence Interval

```
import seaborn as sns
tips = sns.load_dataset('tips')

# Automatically shows mean + 95% CI error bars
sns.barplot(x='day', y='total_bill', data=tips,
            palette='Set2', estimator='mean', errorbar='ci')
```

4.3 Edge Cases & Pitfalls

Problem	Cause	Fix
Y-axis NOT starting at 0	Auto-scale or truncated axis	ALWAYS start bars at 0. Use <code>ax.set_ylim(0, max)</code>
Hard to compare in stacked	Middle segments float	Use grouped bar for precise comparison
3D bar chart	3D looks impressive but distorts perception	Never use 3D bars — use flat 2D always
Too many categories	> 15 bars = cluttered	Keep top-N, group rest as "Other"
Categories are numeric	Bar chart treats them as categories	If numeric+ordered, use line chart instead
Missing category	Category with 0 value not shown	Explicitly include zero-value bars

⚠️ **GOLDEN RULE:** Bar charts must ALWAYS start at zero. Starting at a higher value is a visual lie that exaggerates differences.

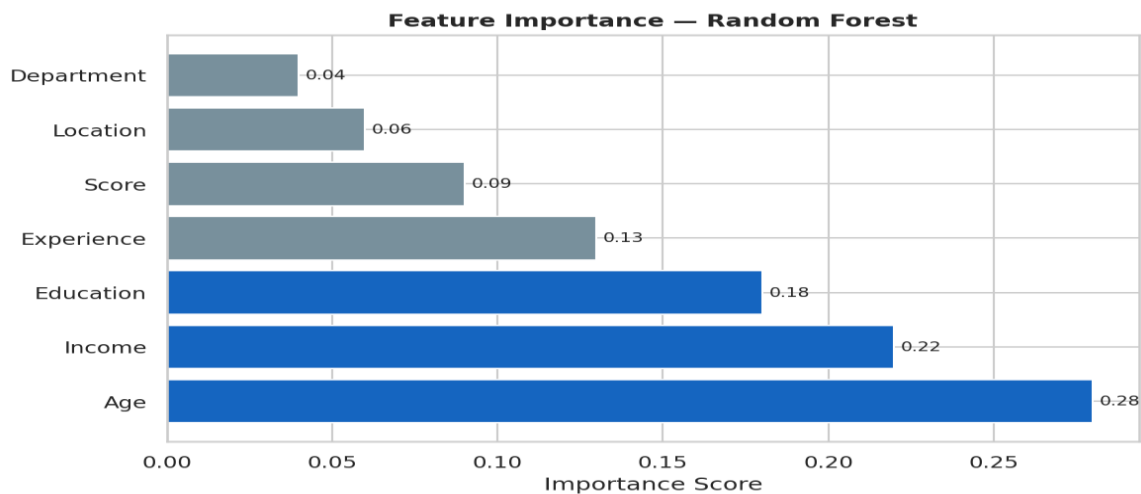
4.4 DS / ML Use Cases

Feature Importance: Horizontal bar chart sorted by importance score — immediately shows which features matter most for your model.

Class Distribution: Show how many samples per class in your dataset. Reveals class imbalance before training.

Model Comparison: Bar chart of accuracy/F1/AUC for multiple models side by side.

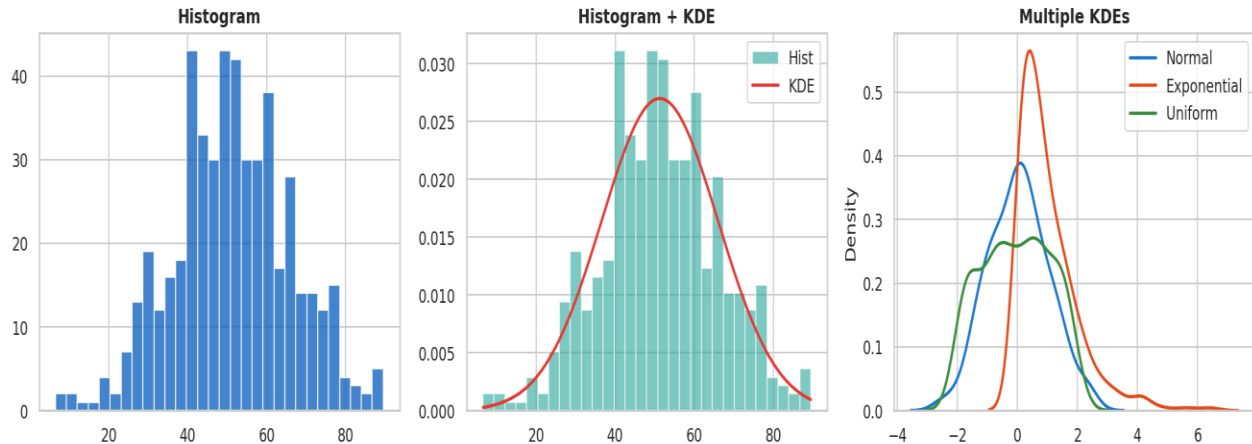
Ablation Studies: Show performance with vs without each component of your model.



5. Histogram & KDE

A histogram groups numeric data into "bins" (ranges) and shows how many data points fall into each bin. It reveals the **shape, spread, and center** of a distribution. Think of it like sorting exam scores into buckets: 0-10, 10-20, 20-30... and counting how many students fall in each.

A **KDE (Kernel Density Estimate)** is a smooth version of a histogram — instead of discrete bars, it gives you a smooth curve showing where the data is concentrated.



5.1 When to Use Histograms

Use Case	Example
Distribution shape	Is data normal, skewed, bimodal?
Outlier detection	Long tail on right = some extreme values
Feature analysis in ML	Check each feature's distribution before modeling
Comparing distributions	Show overlapping histograms for two groups
Residual analysis	Histogram of residuals should be bell-shaped

5.2 Python Code

Histogram with Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

# histplot = modern replacement for distplot
fig, ax = plt.subplots(figsize=(8, 5))
sns.histplot(tips['total_bill'],
             bins=25,      # number of bins
             kde=True,     # overlay smooth KDE curve
             color='#1976D2',
             ax=ax)
ax.set_title('Distribution of Total Bill', fontweight='bold')
plt.tight_layout(); plt.show()
```

Multiple Distributions — Overlapping

```
fig, ax = plt.subplots(figsize=(9, 5))
for group, color in [('Lunch', '#1976D2'), ('Dinner', '#E53935')]:
    subset = tips[tips['time'] == group]['total_bill']
    sns.histplot(subset, kde=True, color=color, alpha=0.5,
                 label=group, ax=ax)
ax.legend(title='Meal Time')
ax.set_title('Bill Distribution: Lunch vs Dinner', fontweight='bold')
```

Choosing the Right Number of Bins

```
# Too few bins = too much smoothing (hides structure)
# Too many bins = noisy (shows random fluctuation)

# Rules of thumb:
# Sturges' Rule: bins = ceil(log2(n) + 1)
# Scott's Rule: bins = 'auto' (matplotlib default)
# Rice Rule: bins = ceil(2 * n**(1/3))

# In matplotlib:
ax.hist(data, bins='auto') # automatic
ax.hist(data, bins=30)     # fixed number
ax.hist(data, bins=np.arange(0, 100, 5)) # custom bin edges
```

5.3 KDE in Detail

KDE places a small "kernel" (like a mini bell curve) at each data point, then adds them all up to get a smooth density curve. The **bandwidth** parameter controls how smooth the curve is — small bandwidth = spiky (overfits), large bandwidth = very smooth (underfits).

KDE with Custom Bandwidth

```
# Using scipy directly for more control
from scipy.stats import gaussian_kde
import numpy as np

data = np.random.normal(0, 1, 500)
kde = gaussian_kde(data, bw_method=0.3) # 0.3 = narrow bandwidth
x_range = np.linspace(-4, 4, 200)
ax.plot(x_range, kde(x_range), lw=2, label='KDE bw=0.3')

# Using seaborn with bw_adjust
sns.kdeplot(data, bw_adjust=0.5, label='narrow') # spiky
sns.kdeplot(data, bw_adjust=2.0, label='smooth') # smooth
```

5.4 Edge Cases & Pitfalls

Problem	Fix
Wrong bin count changes the shape story	Always try 3-4 bin counts and pick the most informative one
KDE extends below zero for positive-only data	Use clip=(0, None) in sns.kdeplot to cut off at 0
Comparing groups with different n	Use density=True (normalize to area=1)
Bimodal shape = two hidden groups	Investigate if there's a categorical variable causing the split
Outliers squish the main distribution	Remove or clip extreme outliers, or use log scale

5.5 DS / ML / DL Use Cases

Before modeling: Check if features are normally distributed (some models assume this). Apply log-transform to right-skewed data.

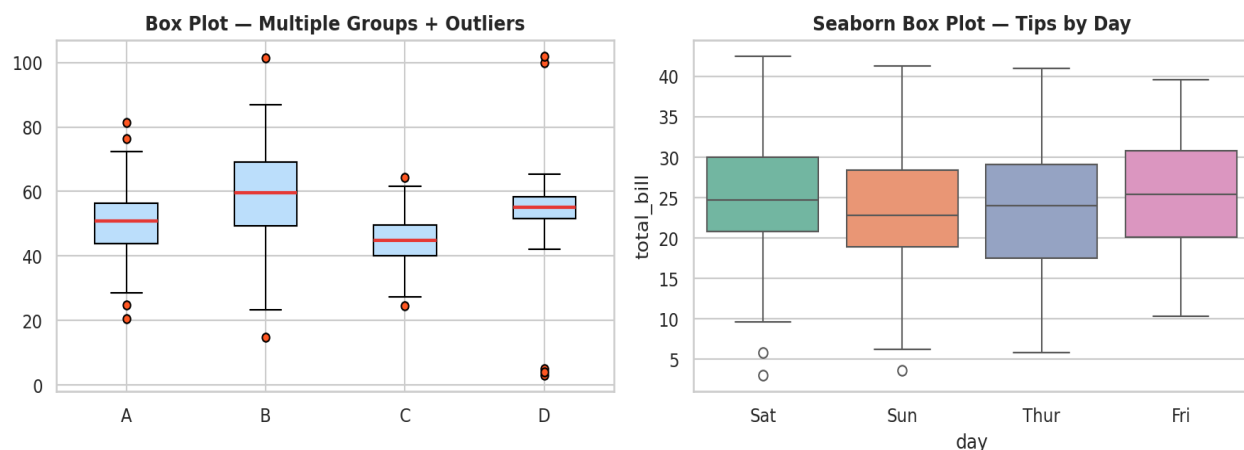
After modeling: Plot histogram of residuals — should be bell-shaped. If skewed, your model is systematically wrong in one direction.

DL: Plot gradient distributions per layer to check for vanishing (all near zero) or exploding (huge values) gradients.

✓PRO TIP: In data preprocessing pipelines, always plot histograms for every numeric feature. This single step reveals outliers, skewness, and distributions in one glance.

6. Box Plot

A box plot (also called a box-and-whisker plot) compresses an entire distribution into a simple diagram showing 5 key statistics at once. It's like a summary statistics report turned into a visual.



6.1 Anatomy of a Box Plot

Component	Statistical Meaning	How to Read It
Bottom of box (Q1)	25th percentile	25% of data falls below this line
Middle line (Q2)	Median (50th percentile)	Half the data is above, half below
Top of box (Q3)	75th percentile	75% of data falls below this line
Box height (IQR)	Interquartile Range = Q3 - Q1	Tall box = data is spread out
Lower whisker	$Q1 - 1.5 \times IQR$	Smallest non-outlier value
Upper whisker	$Q3 + 1.5 \times IQR$	Largest non-outlier value
Dots beyond whiskers	Outliers	Values that are unusually extreme

6.2 Python Code

Matplotlib Box Plot

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
data_a = np.random.normal(50, 10, 100)
data_b = np.random.normal(65, 8, 100)
data_c = np.concatenate([np.random.normal(55, 5, 95), [100, 102, 5, 3]]) # with outliers

fig, ax = plt.subplots(figsize=(8, 6))
bp = ax.boxplot([data_a, data_b, data_c],
                labels=['Group A', 'Group B', 'Group C (outliers)'],
                patch_artist=True,
                boxprops=dict(facecolor='BBDEFB', color='1565C0'),
                medianprops=dict(color='E53935', linewidth=2.5),
                flierprops=dict(marker='o', markerfacecolor='FF5722', markersize=5))
ax.set_title('Box Plot Comparison', fontweight='bold')
plt.tight_layout(); plt.show()
```

Seaborn Box Plot — Grouped by Category

```
import seaborn as sns
tips = sns.load_dataset('tips')

fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(x='day', y='total_bill', data=tips,
            hue='sex', # split each day by gender
            palette='Set2',
            ax=ax)
ax.set_title('Bill Distribution by Day and Gender', fontweight='bold')
plt.tight_layout(); plt.show()
```

6.3 Edge Cases & Pitfalls

Problem	Fix
Median not centered = skewed distribution	Expected behavior — it means data is not symmetric
Many outliers in one group	Investigate those outliers — they might be data errors
All boxes look flat/thin	Data range very small compared to axis scale — zoom in
Groups have very different n	Box plot hides this — add jitter or note sample sizes
Notched box plot confusing	Notches show 95% CI of median — useful for comparing medians

6.4 DS / ML Use Cases

Outlier detection: Before ML preprocessing, box plots quickly show which features have extreme values that might need clipping or normalization.

Model comparison: Show distribution of cross-validation scores across folds — a tight box = stable model.

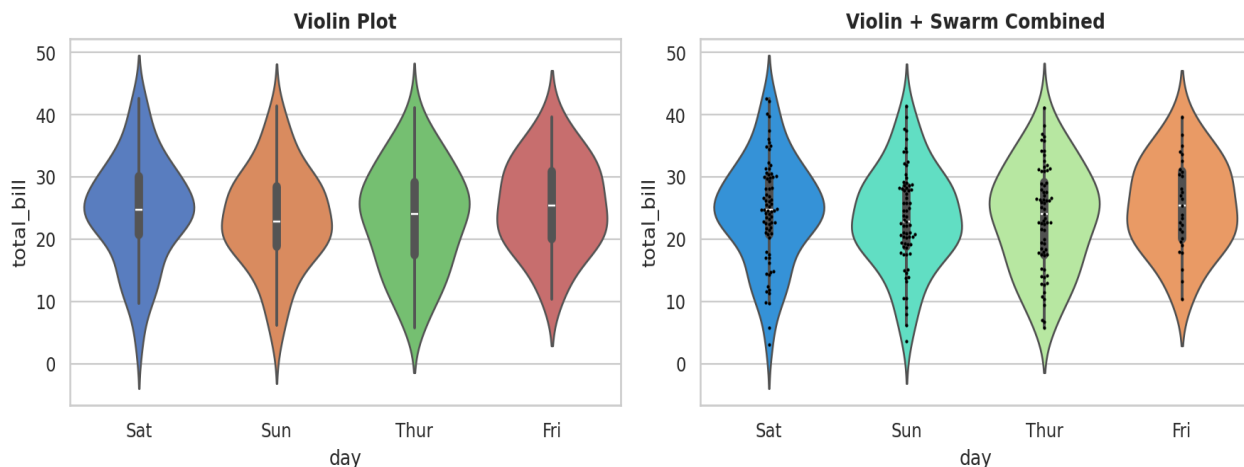
Comparing features: Compare the distribution of a feature across target classes. If a feature's distribution differs significantly between classes, it's a strong predictor.

Box plot vs Violin plot: Box plot shows less detail but is more compact. Violin plot shows full distribution shape. Use violin when you suspect bimodal distributions.

7. Violin Plot

A violin plot is like a box plot, but instead of just showing quartiles, it shows the **full probability density** on both sides. The wider the "belly" of the violin, the more data points exist in that region.

Imagine mirroring a KDE curve on both sides of a vertical axis — that's a violin plot. It reveals shapes that box plots hide, like bimodal distributions (two humps).



7.1 When to Use Violin vs Box Plot

Situation	Use Box Plot	Use Violin Plot
Distribution shape matters	X (hides shape)	✓ (shows full shape)
Bimodal distributions	X (hides two humps)	✓ (clearly shows humps)
Space is limited	✓ (compact)	X (takes more space)
Comparing many groups	✓ (up to 10+ groups)	X (gets crowded > 6)
Showing individual points	X (not standard)	✓ (combine with swarm)
Non-technical audience	✓ (familiar)	X (less familiar)

7.2 Python Code

Violin Plot with Seaborn

```
import seaborn as sns
tips = sns.load_dataset('tips')

fig, ax = plt.subplots(figsize=(9, 6))
sns.violinplot(x='day', y='total_bill', data=tips,
               palette='muted',
               inner='box',      # 'box', 'quart', 'point', 'stick', None
               cut=2,           # extend how far past data range
               ax=ax)
ax.set_title('Violin Plot - Bill by Day', fontweight='bold')
plt.tight_layout(); plt.show()
```

Violin + Swarm (Best of Both Worlds)

```
# Violin shows distribution shape
# Swarm shows all individual data points
```

```
fig, ax = plt.subplots(figsize=(10, 6))

sns.violinplot(x='day', y='total_bill', data=tips,
               palette='rainbow', inner=None, ax=ax)
sns.swarmplot(x='day', y='total_bill', data=tips,
              color='black', size=2.5, alpha=0.7, ax=ax)

ax.set_title('Violin + Swarm - Complete View', fontweight='bold')
plt.tight_layout(); plt.show()
```

7.3 DS / ML Use Cases

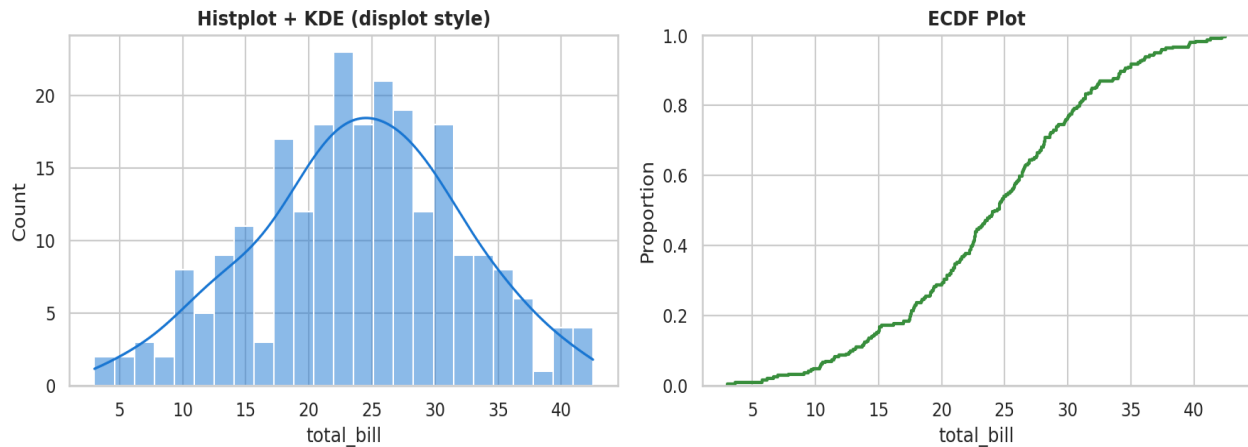
Cross-validation results: Show the distribution of accuracy across folds for each model. A narrow violin = stable, consistent model.

Hyperparameter tuning: Show metric distribution for each hyperparameter setting.

Class-conditional distributions: For each class, show the distribution of a feature. Wide separation between class violins = discriminative feature.

8. Distribution Plots (Seaborn)

Seaborn provides specialized distribution plots: **displot**, **jointplot**, **pairplot**, **rugplot**, and **ECDF**. These are higher-level than raw histograms and automatically combine multiple views.



8.1 displot / histplot

sns.displot() is the modern one-stop-shop for univariate distributions. It can show histograms, KDE, ECDF, and rug marks.

displot and histplot

```
import seaborn as sns
tips = sns.load_dataset('tips')

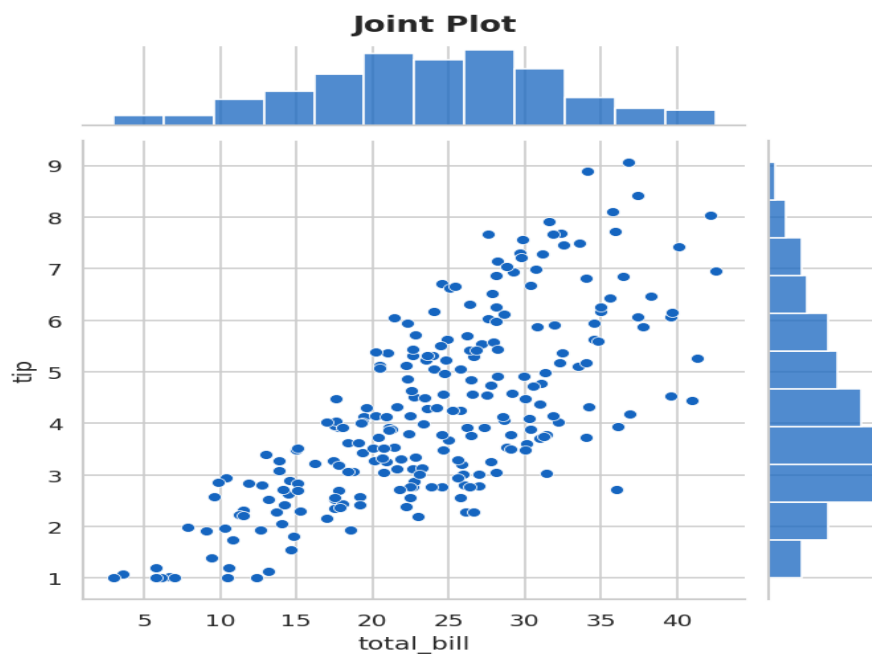
# Figure-level function (creates its own figure)
sns.displot(tips['total_bill'], bins=25, kde=True)

# Axes-level function (works within plt.subplots)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(tips['total_bill'], bins=20, kde=True, ax=axes[0])
sns.histplot(tips['tip'], bins=20, kde=True, ax=axes[1])
plt.tight_layout(); plt.show()

# Multiple distributions on one plot
sns.displot(tips, x='total_bill', hue='time', kind='kde', fill=True)
```

8.2 Joint Plot

A joint plot shows the relationship between two variables (like a scatter plot) AND their individual distributions on the margins simultaneously. This is incredibly useful for exploratory data analysis.



Joint Plot — All Kinds

```
# Scatter + marginal histograms
sns.jointplot(x='total_bill', y='tip', data=tips, kind='scatter')

# KDE contour + marginal KDE
sns.jointplot(x='total_bill', y='tip', data=tips, kind='kde', fill=True)

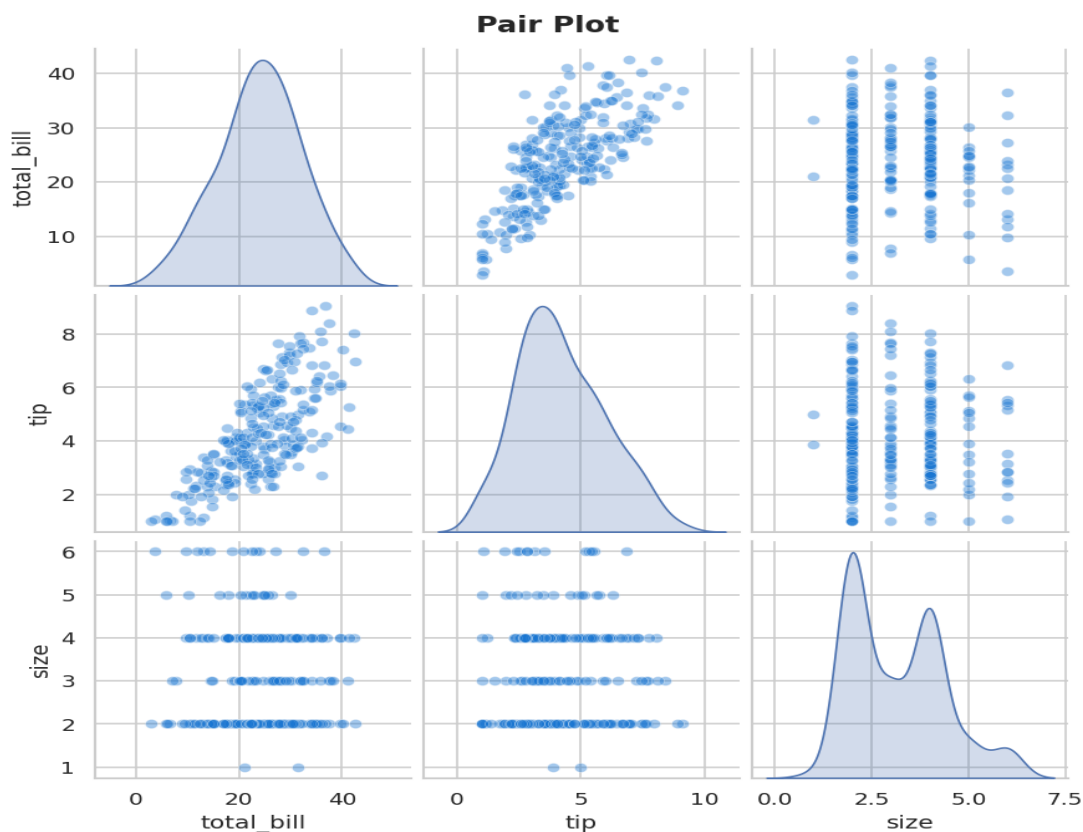
# Hexbin + marginals (for large datasets)
sns.jointplot(x='total_bill', y='tip', data=tips, kind='hex')

# Regression line + marginals
sns.jointplot(x='total_bill', y='tip', data=tips, kind='reg')

# Residuals from regression fit
sns.jointplot(x='total_bill', y='tip', data=tips, kind='resid')
```

8.3 Pair Plot

A pair plot creates a grid of plots where every numeric variable is plotted against every other variable. The diagonal shows each variable's own distribution. This gives you a quick overview of **all pairwise relationships** in your dataset in one shot.

**Pair Plot**

```
import seaborn as sns
tips = sns.load_dataset('tips')

# Basic pair plot
sns.pairplot(tips)

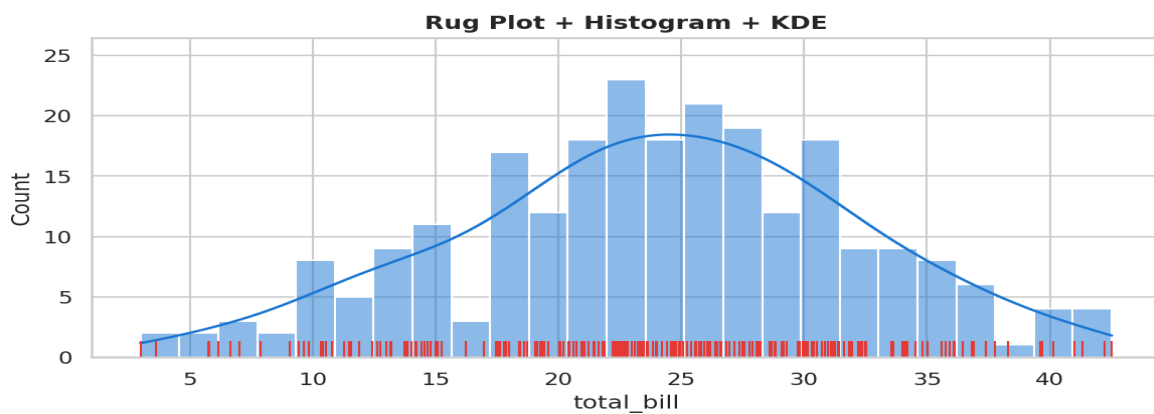
# With hue (color by category)
sns.pairplot(tips, hue='sex', palette='Set1')

# Select specific columns (don't include all if dataset is large!)
sns.pairplot(tips[['total_bill', 'tip', 'size']],
             diag_kind='kde',          # distribution on diagonal
             plot_kws={'alpha': 0.5})
```

⚠ Pair plot is heavy — for large datasets (> 50,000 rows), sample first: `df.sample(5000)` to keep it fast.

8.4 Rug Plot

A rug plot adds small tick marks along the axis showing the exact position of every data point. It is usually combined with a histogram or KDE to show where individual points actually are.

**Rug Plot**

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(tips['total_bill'], bins=25, kde=True, ax=ax, alpha=0.5)
sns.rugplot(tips['total_bill'], ax=ax, height=0.05, color='red')
```

8.5 ECDF Plot

ECDF (Empirical Cumulative Distribution Function) plots the cumulative proportion of data up to each value. Unlike a histogram, it doesn't depend on bin width choices. The Y value at any X tells you: "What fraction of data is $\leq X$?"

ECDF Plot

```
sns.ecdfplot(tips['total_bill'], color='#388E3C', linewidth=2)
ax.set_xlabel('Total Bill')
ax.set_ylabel('Cumulative Proportion')
# e.g. At x=20: y=0.5 means 50% of bills are  $\leq$  $20
```

8.6 DS / ML Use Cases

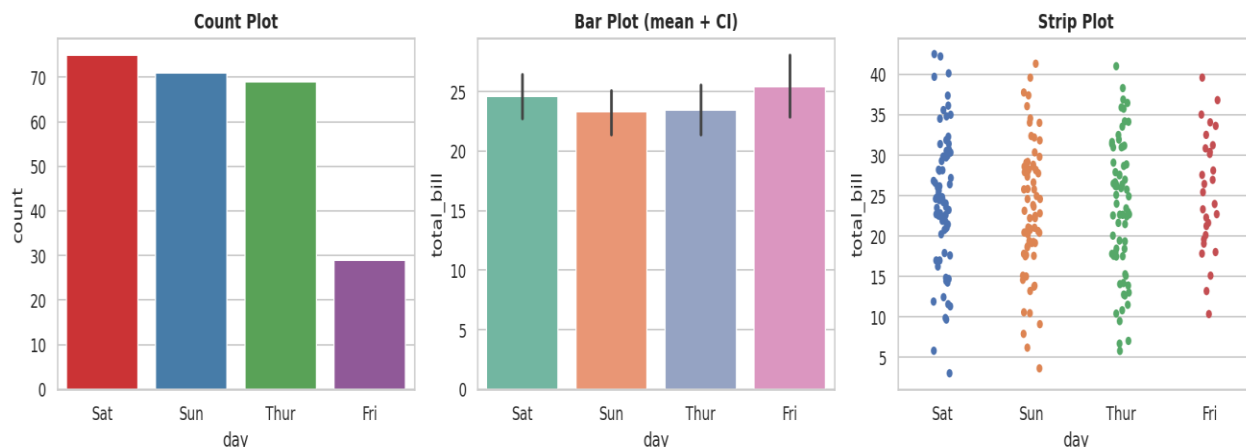
EDA Phase: Pairplot is the single most powerful tool for initial data exploration — run it on every new dataset.

Feature selection: High correlation in pairplot → one feature is redundant → remove to reduce multicollinearity.

Data quality: ECDF quickly shows percentiles — "what value separates the top 10% of customers?"

9. Categorical Plots (Seaborn)

Categorical plots are specifically designed to visualize relationships between a **categorical variable** (like day of week, product type, country) and a **numeric variable**. Seaborn provides a rich set of categorical visualizations.



9.1 Count Plot

Count plot simply counts how many times each category appears in the data. It is a bar chart of frequencies, not values.

Count Plot

```
import seaborn as sns
tips = sns.load_dataset('tips')

# Count of each day
sns.countplot(x='day', data=tips, palette='Set1')

# With hue (split by smoker)
sns.countplot(x='day', data=tips, hue='smoker', palette='rainbow')

# Sort by frequency
order = tips['day'].value_counts().index
sns.countplot(x='day', data=tips, order=order, palette='viridis')
```

9.2 Bar Plot (Seaborn)

Seaborn's barplot shows the **mean** of a numeric variable for each category, with **95% confidence interval error bars** automatically added. It is more statistically informative than matplotlib's plain bar chart.

Seaborn Bar Plot with Confidence Interval

```
# Shows mean total_bill per day, with CI bars
sns.barplot(x='day', y='total_bill', data=tips, palette='Set2')

# Change estimator function
import numpy as np
sns.barplot(x='sex', y='total_bill', data=tips,
            estimator=np.sum, # show total, not mean
            palette='Set1')

# With hue
sns.barplot(x='day', y='tip', data=tips,
```

```
hue='sex', palette='husl')
```

9.3 Strip Plot

A strip plot shows all individual data points as dots, arranged by category. Adding jitter spreads overlapping points horizontally so you can see them all.

Strip Plot

```
sns.stripplot(x='day', y='total_bill', data=tips,
              jitter=True, # spread overlapping points
              alpha=0.6,
              palette='deep')
```

9.4 Swarm Plot

A swarm plot is like a strip plot but smarter — it positions dots so that no two dots overlap. This gives you a clear view of every single data point AND the distribution shape. Great for small to medium datasets.

Swarm Plot (and combining with Violin)

```
# Pure swarm
sns.swarmplot(x='day', y='total_bill', data=tips, palette='PiYG')

# Combined: violin (distribution) + swarm (individual points)
sns.violinplot(x='day', y='total_bill', data=tips,
               palette='rainbow', inner=None)
sns.swarmplot(x='day', y='total_bill', data=tips,
               color='black', size=2.5)
```

9.5 Point Plot

A point plot shows the mean value per category as a dot, connected by a line, with confidence interval bars. It is the best choice for **showing interaction effects** between two categorical variables.

Point Plot

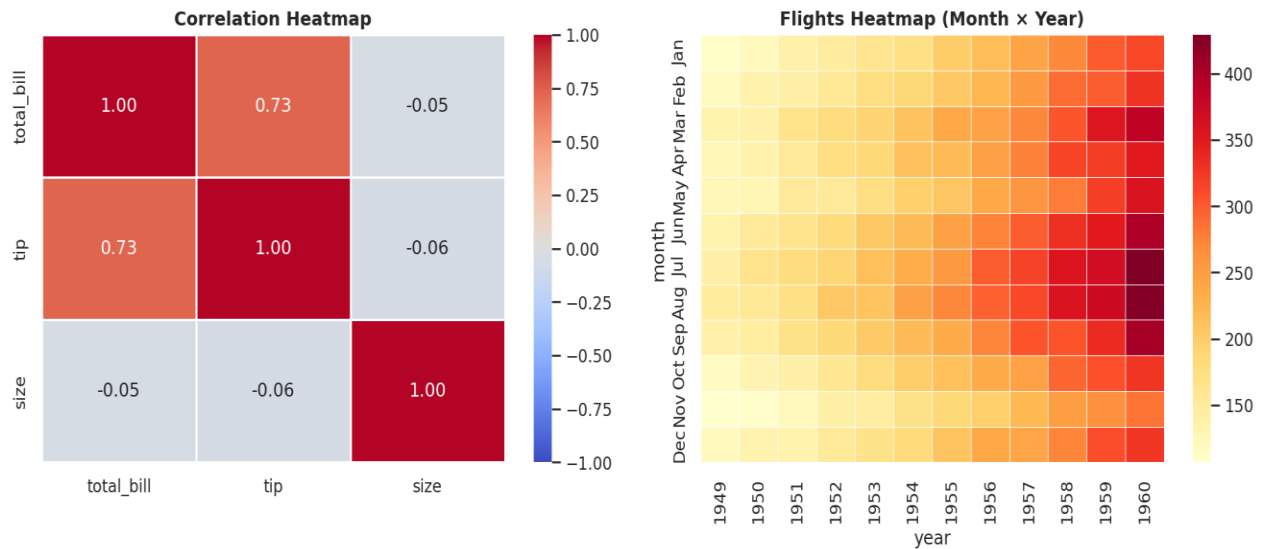
```
sns.pointplot(x='day', y='total_bill', data=tips,
              hue='sex', # show separate lines per gender
              palette='Set1',
              markers=['o', 's'], # different markers per group
              linestyle=['-', '--'])
```

9.6 DS / ML Use Cases

Plot	ML / DS Application
Count Plot	Check class imbalance in classification datasets
Bar Plot	Compare model accuracy across different architectures
Strip / Swarm	Show per-sample metric values across categories
Point Plot	Interaction effects in A/B testing; show metric vs hyperparameter setting
Box + Cat	Feature value distribution per class (is this feature discriminative?)

10. Matrix Plots (Heatmaps)

Matrix plots visualize 2D data (a grid of values) using color intensity. The brighter or darker a cell, the higher or lower its value. Heatmaps are incredibly powerful for showing **correlation matrices**, **confusion matrices**, **pivot tables**, and **any grid-structured data**.



10.1 Correlation Heatmap

The most common use: visualize a correlation matrix. Each cell shows the correlation (−1 to 1) between two variables. Red = high positive correlation, Blue = negative correlation.

Correlation Heatmap

```
import seaborn as sns
tips = sns.load_dataset('tips')

# Calculate correlation matrix
corr = tips[['total_bill', 'tip', 'size']].corr()

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr,
            annot=True,          # show numbers in each cell
            fmt='.2f',          # 2 decimal places
            cmap='coolwarm',    # diverging colormap (good for correlation)
            vmin=-1, vmax=1,    # fix color range to [-1, 1]
            linewidths=0.5,     # cell borders
            square=True,        # make cells square
            ax=ax)
ax.set_title('Correlation Matrix', fontweight='bold')
plt.tight_layout(); plt.show()
```

10.2 Flights Pivot Heatmap

Heatmap from Pivot Table

```
flights = sns.load_dataset('flights')

# Reshape: rows=month, columns=year, values=passengers
pvt = flights.pivot_table(values='passengers',
                          index='month',
                          columns='year')
```

```
sns.heatmap(pvt,
             cmap='YlOrRd',  # sequential colormap (for counts)
             annot=True,
             fmt='d',
             linewidths=0.3)
```

10.3 Cluster Map

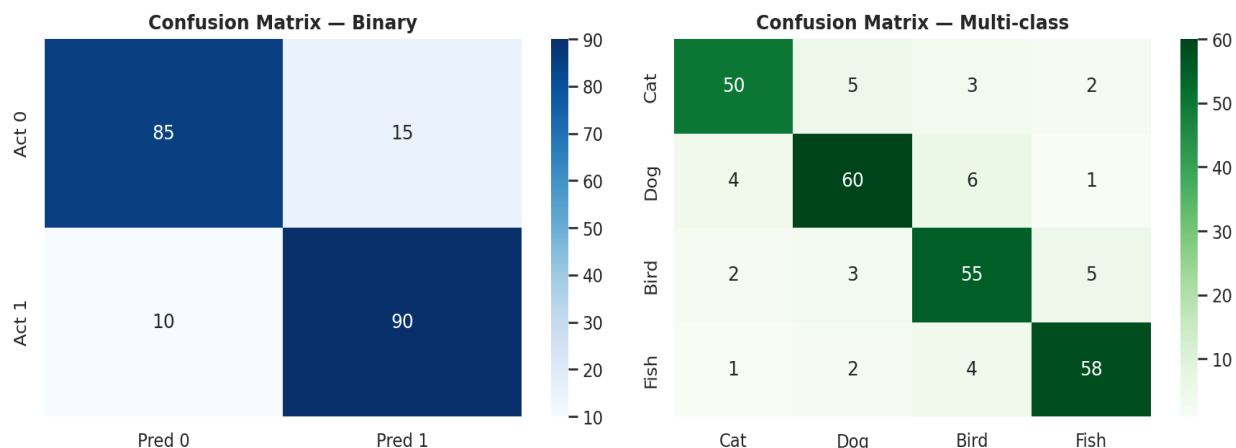
A cluster map is a heatmap with hierarchical clustering applied to both rows and columns, so that similar rows/columns are placed next to each other.

Cluster Map

```
sns.clustermap(corr_matrix,
               cmap='coolwarm', annot=True,
               figsize=(10, 8),
               row_cluster=True,  # cluster rows
               col_cluster=True) # cluster columns
```

10.4 Confusion Matrix Heatmap

A confusion matrix heatmap is the standard way to evaluate classification models. Each row = actual class, each column = predicted class. The diagonal = correct predictions.



Confusion Matrix — ML Model Evaluation

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_true = [0, 1, 0, 1, 1, 0, 1, 0]
y_pred = [0, 1, 1, 1, 0, 0, 1, 0]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred 0', 'Pred 1'],
            yticklabels=['Act 0', 'Act 1'])
plt.title('Confusion Matrix', fontweight='bold')
plt.ylabel('Actual'); plt.xlabel('Predicted')
```

10.5 Edge Cases & Pitfalls

Problem	Fix
Color scale misleading	For correlation, always set vmin=-1, vmax=1
Too many features = cluttered heatmap	Use cluster map to group similar features; or show only top correlated pairs
Sequential vs diverging colormap confusion	Use coolwarm/RdBu for diverging data (correlation); YlOrRd/Blues for counts
Annotations too small	Reduce number of features or increase figure size
Duplicate correlation (symmetric matrix)	Use mask=np.triu(np.ones_like(corr)) to show only lower triangle

10.6 DS / ML / DL / CV Use Cases

Feature selection: High correlation (> 0.9) between two features means one is redundant — drop it to reduce multicollinearity.

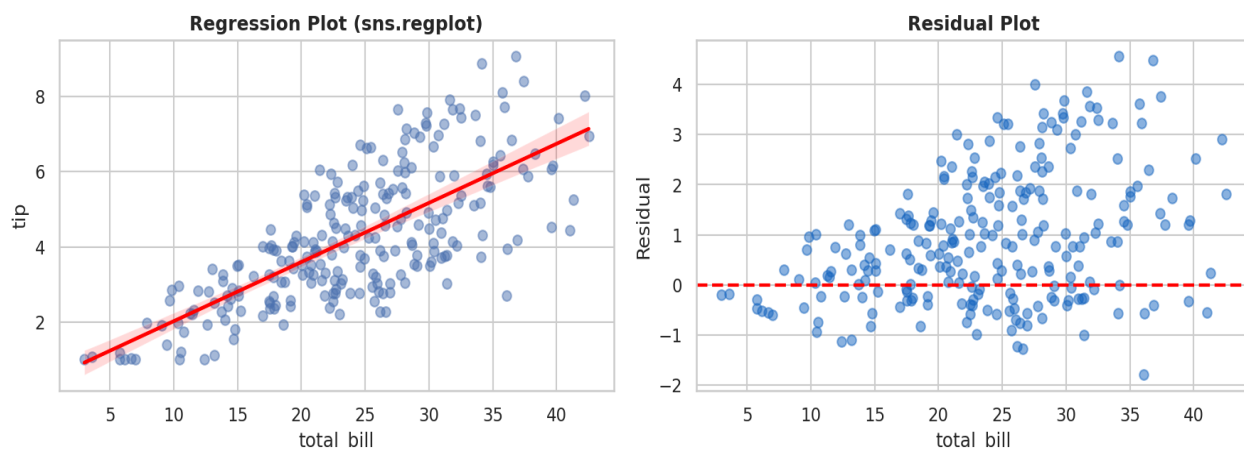
Model evaluation: Confusion matrix is the single most important visualization for classification models — it shows exactly where the model makes errors.

CV (Computer Vision): Attention maps in Transformers are visualized as heatmaps — shows which input tokens the model focused on.

DL: Weight matrices and activation patterns across layers visualized as heatmaps.

11. Regression Plots

Regression plots show the relationship between two numeric variables AND overlay a regression line (the best-fit line) to show the trend. They immediately show **how strongly correlated two variables are** and in what direction.



11.1 sns.regplot — Single Regression

regplot — Simple Linear Regression Overlay

```
import seaborn as sns
tips = sns.load_dataset('tips')

fig, ax = plt.subplots(figsize=(8, 6))
sns.regplot(x='total_bill', y='tip', data=tips,
            scatter_kws={'alpha': 0.5, 'color': '#1976D2'},
            line_kws={'color': 'red', 'linewidth': 2},
            ci=95,      # show 95% confidence interval band
            ax=ax)
ax.set_title('Total Bill vs Tip (with regression)', fontweight='bold')
plt.tight_layout(); plt.show()
```

11.2 sns.lmplot — Faceted Regression

lmplot is the figure-level version of regplot. It can split the regression by a categorical variable using hue, col, or row — you get separate regression lines per group.

lmplot — Separate Lines Per Group

```
# Separate regression line per day
sns.lmplot(x='total_bill', y='tip', data=tips,
          hue='day',      # different color + line per day
          palette='deep',
          markers='o',
          scatter_kws={'s': 50},
          ci=95,
          aspect=1.4)

# Separate plots per time
sns.lmplot(x='total_bill', y='tip', data=tips,
          col='time',     # separate column per time (Lunch/Dinner)
          hue='sex')
```

11.3 Residual Plot

A residual plot shows the errors (actual – predicted) on the Y axis vs the fitted values (or X) on the X axis. A well-behaved model should show **randomly scattered residuals around zero** with no pattern. Any visible pattern means the model is missing something.

Residual Plot

```
# Using seaborn
sns.residplot(x='total_bill', y='tip', data=tips, lowess=True,
              scatter_kws={'alpha': 0.5},
              line_kws={'color': 'red', 'linewidth': 2})
ax.axhline(0, color='black', linewidth=1)

# Manually
from sklearn.linear_model import LinearRegression
import numpy as np

X = tips[['total_bill']]
y = tips['tip']
model = LinearRegression().fit(X, y)
residuals = y - model.predict(X)

plt.scatter(model.predict(X), residuals, alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
```

11.4 DS / ML Use Cases

OLS Regression Analysis: Check if linear regression assumptions hold — scatter should be random around zero in residual plot.

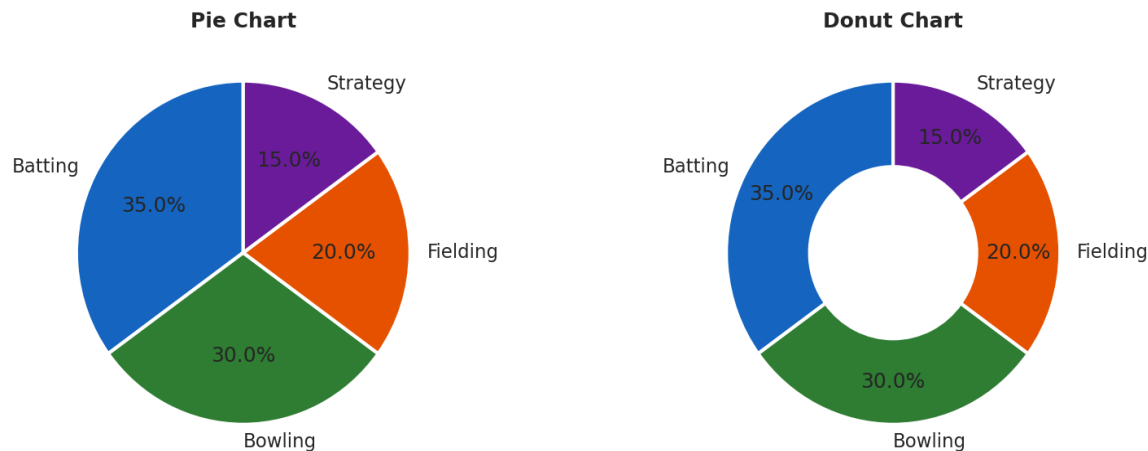
Feature Engineering: If regplot shows a curve (not line), try log-transforming the feature.

Predicted vs Actual Plot: For any regression model, scatter plot of predicted vs actual values — perfect model = all points on diagonal.

⚠ If residuals show a fan-shaped pattern (increasing variance), you have heteroscedasticity — try log-transforming the target variable.

12. Pie Chart & Donut Chart

Pie charts show **parts of a whole** — each slice represents a percentage of the total. Donut charts are just pie charts with the center cut out, which makes them easier to read (the center can also hold a summary number).



12.1 When to Use (and NOT Use) Pie Charts

Use Pie Chart	Avoid Pie Chart
3-5 categories at most	More than 5-6 slices (too hard to compare)
Proportions are very different (e.g., 70% vs 30%)	When differences are subtle (e.g., 24% vs 26%)
Showing part-to-whole is the main point	When you need to compare exact values across groups
Non-technical audience	Scientific/analytical reports (use bar chart instead)

12.2 Python Code

```
Pie and Donut Charts
import matplotlib.pyplot as plt

labels = ['Revenue', 'COGS', 'Marketing', 'R&D']
sizes = [45, 30, 15, 10]
colors = ['#1565C0', '#2E7D32', '#E65100', '#6A1B9A']
explode = [0.05, 0, 0, 0] # "explode" the first slice slightly

# Pie Chart
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].pie(sizes, labels=labels, colors=colors,
            autopct='%1.1f%%', # show % on each slice
            startangle=90, # start from top
            explode=explode,
            wedgeprops=dict(edgecolor='white', linewidth=2))
axes[0].set_title('Budget Breakdown', fontweight='bold')

# Donut Chart (just add width parameter)
axes[1].pie(sizes, labels=labels, colors=colors,
            autopct='%1.1f%%', startangle=90, pctdistance=0.75,
```

```
wedgeprops=dict(width=0.5, edgecolor='white', linewidth=2))
axes[1].text(0, 0, '100%', ha='center', va='center',
            fontsize=18, fontweight='bold') # center text
axes[1].set_title('Budget Donut', fontweight='bold')
plt.tight_layout(); plt.show()
```

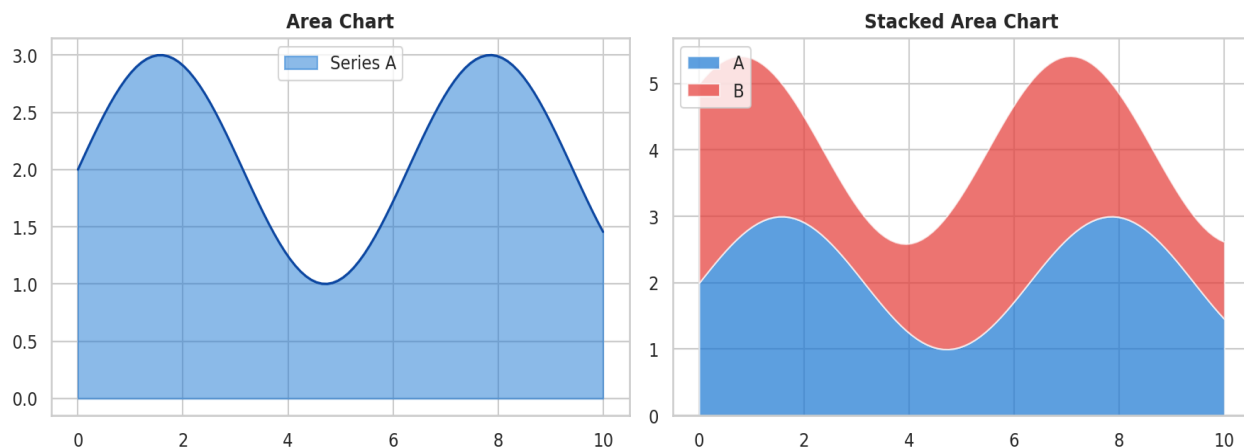
12.3 Edge Cases & Pitfalls

⚠ Pie charts are among the most misused chart types. When in doubt, use a horizontal bar chart — it allows easier comparisons and handles more categories.

Problem	Fix
Slices don't add to 100%	Double-check data; normalize if needed
Percentages show wrong values	Verify data isn't already percentages (would double-count)
3D pie chart requested	Always reject — 3D distorts slice areas. Use flat 2D only.
Too many thin slices	Group small categories into "Other"

13. Area Chart

An area chart is a line chart where the area below the line is filled with color. A stacked area chart shows multiple series stacked on top of each other, useful for showing how the composition of a total changes over time.



13.1 Python Code

Area and Stacked Area Charts

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y1 = np.sin(x) + 2
y2 = np.cos(x) + 3

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Simple area chart
axes[0].fill_between(x, y1, alpha=0.5, color='#1976D2', label='Series A')
axes[0].plot(x, y1, color='#0D47A1', linewidth=1.5) # line on top
axes[0].set_title('Area Chart', fontweight='bold'); axes[0].legend()

# Stacked area chart
axes[1].stackplot(x, [y1, y2], labels=['A', 'B'],
                  colors=['#1976D2', '#E53935'], alpha=0.7)
axes[1].legend(loc='upper left')
axes[1].set_title('Stacked Area Chart', fontweight='bold')

plt.tight_layout(); plt.show()
```

13.2 DS / ML Use Cases

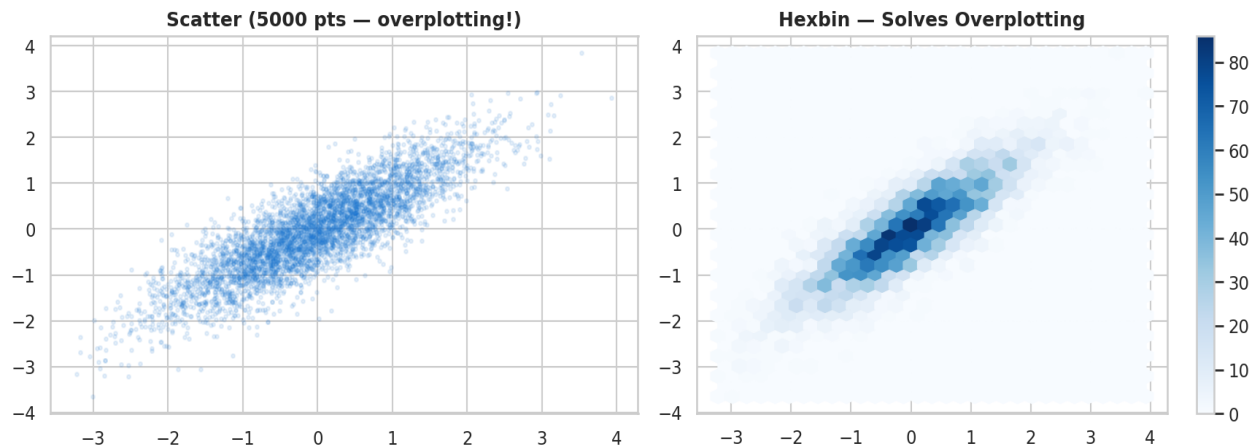
Time-series decomposition: Stacked area chart showing trend + seasonality + residual components.

Resource utilization: CPU / GPU / memory usage over training time — each stacked layer is one component.

Category proportions over time: How market share changed across competitors year by year.

14. Hexbin Plot

A hexbin plot divides the plotting area into hexagonal bins and counts how many data points fall into each hexagon. The darker (or brighter) the hexagon, the more points it contains. It is the solution to **overplotting** — when you have so many points in a scatter plot that they cover each other.



14.1 Python Code

Hexbin Plot

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
x = np.random.randn(10000)
y = x * 0.8 + np.random.randn(10000) * 0.5

fig, ax = plt.subplots(figsize=(8, 6))
hb = ax.hexbin(x, y,
               gridsize=40,      # number of hexagons per axis
               cmap='Blues',      # colormap
               mincnt=1)          # hide empty hexagons

plt.colorbar(hb, ax=ax, label='Count')
ax.set_title('Hexbin - 10,000 Points', fontweight='bold')
plt.tight_layout(); plt.show()
```

14.2 DS / ML Use Cases

Large dataset EDA: When working with millions of rows, scatter plots are unreadable. Hexbin shows the density structure clearly.

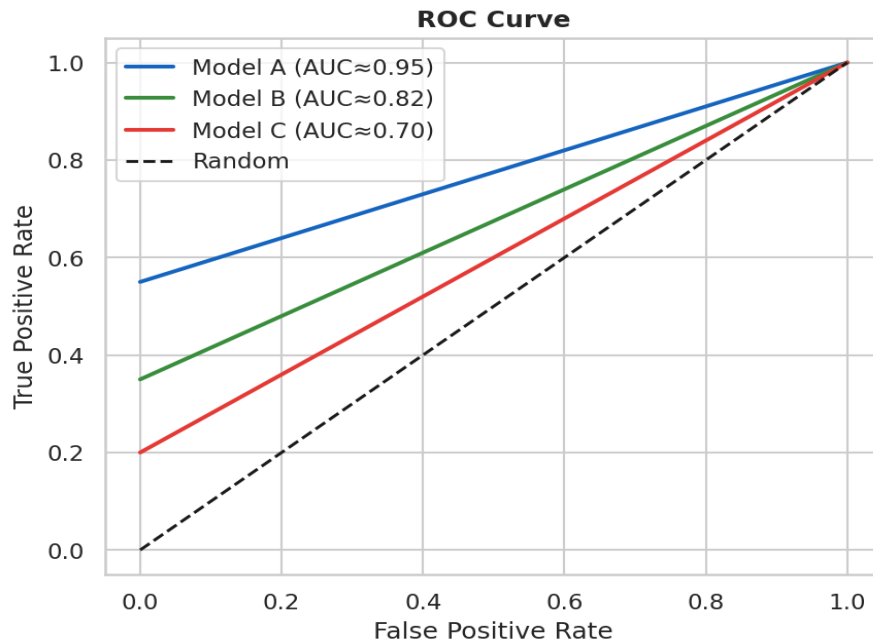
Predicted vs Actual: For large regression datasets, hexbin shows where the model is most accurate (dark hexagons on the diagonal).

✓ Use hexbin whenever your scatter plot has > 5,000–10,000 points and you can't see the distribution clearly.

15. ROC Curve & AUC

The ROC (Receiver Operating Characteristic) curve plots **True Positive Rate (Recall)** on the Y-axis against **False Positive Rate** on the X-axis at every possible classification threshold. It summarizes the performance of a binary classifier across all thresholds.

The **AUC (Area Under the Curve)** is a single number — the area under the ROC curve. AUC = 1.0 means perfect classifier. AUC = 0.5 means random (diagonal line). AUC = 0.0 means perfectly wrong.



15.1 Python Code

ROC Curve with sklearn

```
from sklearn.metrics import roc_curve, auc
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt

# Train a model
model = LogisticRegression()
model.fit(X_train, y_train)
y_proba = model.predict_proba(X_test)[:, 1]

# Calculate ROC
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color='#1565C0', lw=2, label=f'AUC = {roc_auc:.3f}')
ax.plot([0,1],[0,1], 'k--', lw=1.5, label='Random Classifier')
ax.fill_between(fpr, tpr, alpha=0.1, color='#1565C0') # shade AUC
ax.set_xlabel('False Positive Rate (FPR)')
ax.set_ylabel('True Positive Rate (TPR = Recall)')
ax.set_title('ROC Curve', fontweight='bold')
ax.legend(); plt.tight_layout(); plt.show()
```

✓ When comparing multiple models, plot all ROC curves on the same axes. The model with the highest AUC (and the curve furthest to the upper-left) is the best.

15.2 DS / ML Use Cases

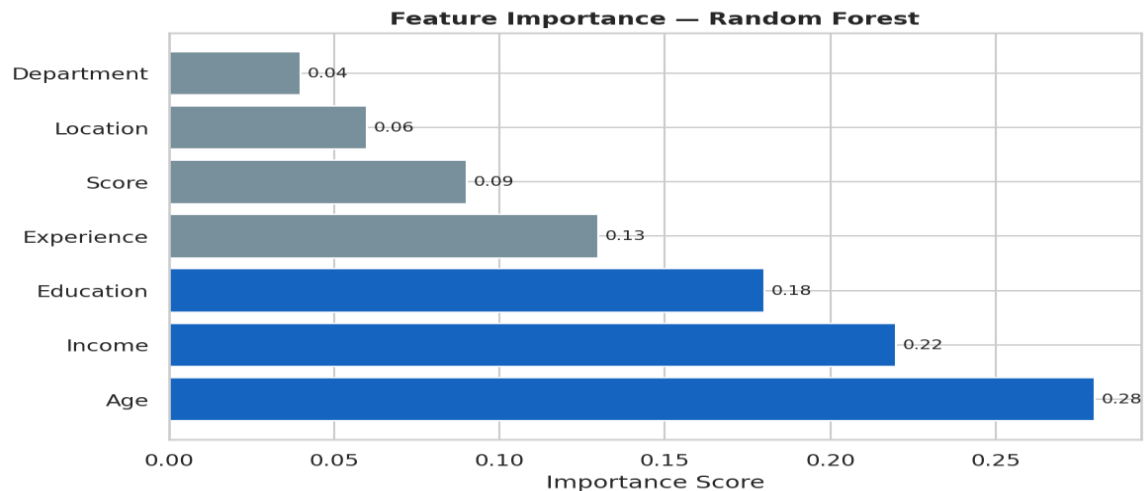
Model selection: Compare ROC curves of different classifiers — Logistic Regression vs SVM vs Random Forest.

Threshold tuning: Choose the threshold that balances FPR/TPR based on your business cost (false positives vs false negatives).

Imbalanced data: Unlike accuracy, AUC is robust to class imbalance.

16. Feature Importance Plot

A feature importance plot shows which input variables (features) had the most influence on a model's predictions. It is almost always shown as a **horizontal bar chart** sorted by importance score, making it easy to read even with many features.



16.1 Python Code

```
Feature Importance from Random Forest
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import matplotlib.pyplot as plt

# Train model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Get importance scores
importances = model.feature_importances_
features = X_train.columns

# Sort and plot
fi = pd.Series(importances, index=features).sort_values()
fig, ax = plt.subplots(figsize=(9, 6))
fi.plot(kind='barh', ax=ax, color='#1565C0')
ax.set_xlabel('Feature Importance Score')
ax.set_title('Random Forest Feature Importance', fontweight='bold')

# Add value labels
for i, v in enumerate(fi):
    ax.text(v + 0.002, i, f'{v:.3f}', va='center', fontsize=9)

plt.tight_layout(); plt.show()
```

16.2 DS / ML / DL Use Cases

Feature selection: Remove features with near-zero importance to reduce overfitting and speed up inference.

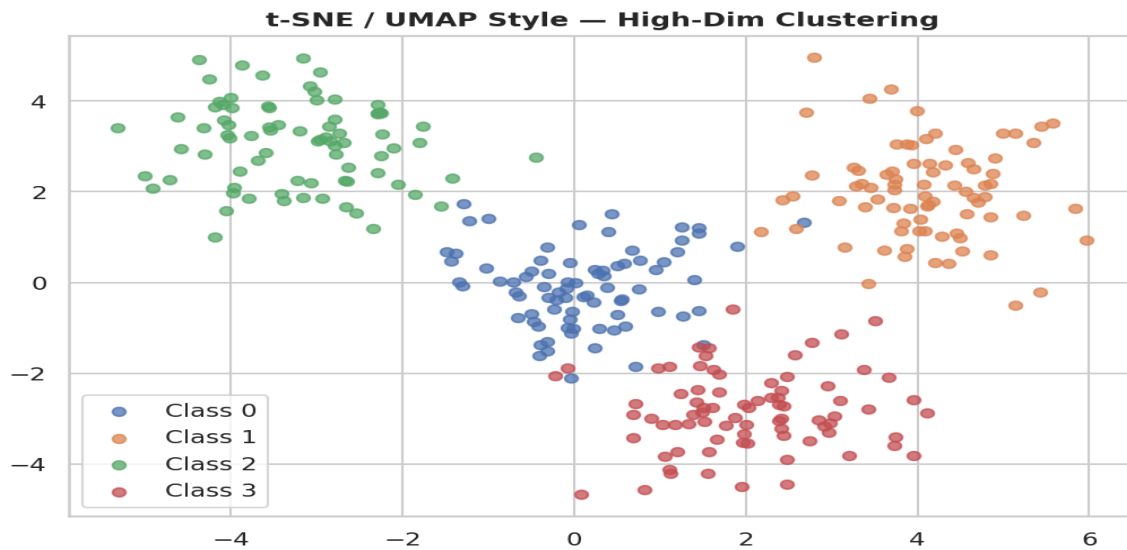
Model debugging: If an obviously irrelevant feature has high importance, there may be a data leakage bug.

DL/SHAP: For neural networks, use SHAP values — plot a SHAP summary (beeswarm) instead of simple bar chart for richer insights.

17. t-SNE / UMAP Visualization

t-SNE and UMAP are dimensionality reduction algorithms. They take **high-dimensional data** (like 512-dimensional image embeddings or 768-dimensional text embeddings) and compress it to 2D so you can visualize it as a scatter plot.

If samples of the same class cluster together in the 2D projection, your model has learned good representations. Overlapping clusters mean poor separation.



17.1 Python Code

t-SNE Visualization

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import numpy as np

# Assume X = (N, 512) high-dimensional features, y = class labels
X = np.random.randn(500, 512) # example: 500 samples, 512 features
y = np.random.choice([0,1,2,3], 500)

# Reduce to 2D
tsne = TSNE(n_components=2, random_state=42,
            perplexity=30, # typical: 5-50
            max_iter=1000)
X_2d = tsne.fit_transform(X)

# Plot
fig, ax = plt.subplots(figsize=(9, 7))
colors = ['#1565C0', '#E53935', '#388E3C', '#F57F17']
for cls in range(4):
    mask = (y == cls)
    ax.scatter(X_2d[mask, 0], X_2d[mask, 1],
               c=colors[cls], label=f'Class {cls}', alpha=0.7, s=30)
ax.legend(title='Class')
ax.set_title('t-SNE — 512D → 2D Projection', fontweight='bold')
plt.tight_layout(); plt.show()
```

i t-SNE: Run only on a sample (< 10,000 points) as it is slow ($O(n^2)$). UMAP is faster and better preserves global structure.

17.2 DS / ML / DL / CV Use Cases

CV: After training a CNN, extract penultimate layer features and apply t-SNE — well-trained models show clearly separated clusters per class.

NLP: Visualize word embeddings or sentence embeddings — semantically similar words cluster together.

Anomaly detection: Normal data points form a tight cluster; anomalies appear as isolated dots far from the cluster.

18. Plotly — Interactive Plots

Plotly creates **interactive** charts that you can zoom, pan, hover, and click. Unlike Matplotlib, Plotly outputs HTML — perfect for dashboards, web apps (Dash, Streamlit), and Jupyter notebooks where you want to explore data dynamically.

18.1 Plotly Express — Quick Plots

Plotly Express — One Line Interactive Charts

```
import plotly.express as px
import seaborn as sns

tips = sns.load_dataset('tips')

# Interactive scatter
fig = px.scatter(tips, x='total_bill', y='tip',
                 color='day', hover_data=['sex', 'smoker'],
                 title='Interactive Scatter - hover for details')
fig.show()

# Interactive bar
fig = px.bar(tips.groupby('day')['total_bill'].mean().reset_index(),
             x='day', y='total_bill', title='Mean Bill by Day')
fig.show()

# Interactive line
fig = px.line(df, x='date', y='value', color='category',
              title='Time Series - Zoomable')
fig.show()
```

18.2 Cufflinks — Pandas + Plotly

Cufflinks connects Pandas DataFrames directly to Plotly. You can call `.iplot()` on any DataFrame column exactly like `.plot()`.

Cufflinks One-Liners

```
import cufflinks as cf
cf.go_offline() # run offline in Jupyter

# Line chart for a single column
tips['total_bill'].iplot(color='blue', title='Total Bills')

# Bar chart from grouped data
tips.groupby('day')['tip'].mean().iplot(kind='bar', title='Mean Tip by Day')

# Scatter plot
tips.iplot(kind='scatter', x='total_bill', y='tip', mode='markers')

# Box plot
tips[['total_bill', 'tip']].iplot(kind='box')
```

18.3 Plotly Graph Objects — Full Control

Custom Plotly Chart (Graph Objects)

```
import plotly.graph_objects as go

fig = go.Figure()
```

```
fig.add_trace(go.Bar(
    x=['A', 'B', 'C', 'D'],
    y=[10, 25, 7, 18],
    marker_color=['#1565C0', '#E53935', '#388E3C', '#F57F17'],
    text=[10, 25, 7, 18],
    textposition='outside'
))
fig.update_layout(
    title='Custom Interactive Bar Chart',
    xaxis_title='Category',
    yaxis_title='Value',
    template='plotly_white'
)
fig.show()
```

i Use Plotly when building dashboards (Dash/Streamlit) or when stakeholders need to explore data interactively. For static reports and papers, use Matplotlib/Seaborn.

19. Working with Images in Matplotlib

Matplotlib can load and display images, which is critical in Computer Vision (CV) workflows. Images in NumPy are just 3D arrays: (height, width, channels). Every pixel operation becomes an array operation.

19.1 Loading and Displaying Images

Load, Display, Crop Images

```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import numpy as np

# Load image (returns NumPy array: shape = H x W x 3 for RGB)
img = mpimg.imread('flower.jpg') # or use PIL/OpenCV
print(img.shape) # e.g., (480, 640, 3)
print(img.dtype) # uint8 (values 0-255)

# Display
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(img)
axes[0].axis('off')
axes[0].set_title('Original Image')

# Crop: img[row_start:row_end, col_start:col_end]
cropped = img[50:200, 100:300]
axes[1].imshow(cropped)
axes[1].axis('off')
axes[1].set_title('Cropped')

# Grayscale
gray = np.mean(img, axis=2).astype(np.uint8)
axes[2].imshow(gray, cmap='gray')
axes[2].axis('off')
axes[2].set_title('Grayscale')

plt.tight_layout(); plt.show()
```

Channel Manipulation

```
# Separate RGB channels
r, g, b = img[:, :, 0], img[:, :, 1], img[:, :, 2]

# Show only red channel
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, channel, title, cmap in zip(axes,
    [img, r, g, b], ['RGB', 'Red', 'Green', 'Blue'],
    [None, 'Reds', 'Greens', 'Blues']):
    ax.imshow(channel, cmap=cmap); ax.axis('off'); ax.set_title(title)
```

19.2 CV Use Cases

Model debugging: Display images alongside model predictions. Show side-by-side: original, ground truth mask, predicted mask.

Data augmentation preview: Before training, visualize augmented samples to ensure augmentation is correct.

Activation map visualization: Show which areas of an image a CNN focuses on (Grad-CAM heatmaps).

Batch visualization: Display a 4×4 or 8×8 grid of training samples to inspect dataset quality.

Image Grid (Batch Visualization)

```
def show_image_grid(images, labels, n=16):  
    """Show n images in a grid with their labels."""  
    fig, axes = plt.subplots(4, 4, figsize=(12, 12))  
    for i, ax in enumerate(axes.flat):  
        if i < n:  
            ax.imshow(images[i])  
            ax.set_title(f'Label: {labels[i]}', fontsize=8)  
            ax.axis('off')  
    plt.suptitle('Training Batch Sample', fontweight='bold')  
    plt.tight_layout(); plt.show()
```

20. Axis, Titles & Professional Styling

Great data always gets overlooked when the chart looks amateur. Professional styling is not about making things pretty — it's about making your data readable, trustworthy, and publication-ready.

20.1 Complete Styling Template

Professional Plot Template

```
import matplotlib.pyplot as plt
import seaborn as sns

# — 1. Set global style —————
plt.rcParams.update({
    'font.family':      'Arial',
    'font.size':        11,
    'axes.titlesize':   14,
    'axes.titleweight': 'bold',
    'axes.labelsize':   12,
    'xtick.labelsize':  10,
    'ytick.labelsize':  10,
    'legend.fontsize':  10,
    'figure.dpi':       150,
    'axes.spines.top':   False, # remove top border
    'axes.spines.right': False, # remove right border
    'axes.grid':         True,
    'grid.alpha':        0.35,
    'grid.linestyle':    '--',
})

# — 2. Create figure —————
fig, ax = plt.subplots(figsize=(10, 6))

# — 3. Your plot —————
# ax.plot(...) or ax.bar(...) etc.

# — 4. Title, axis labels —————
ax.set_title('Main Title Here', fontsize=15, fontweight='bold', pad=15)
ax.set_xlabel('X Variable (units)', fontsize=12, labelpad=10)
ax.set_ylabel('Y Variable (units)', fontsize=12, labelpad=10)

# — 5. Legend —————
legend = ax.legend(title='Group Variable', loc='upper left',
                  framealpha=0.9, edgecolor='#BFCDD8')
legend.get_title().set_fontweight('bold')

# — 6. Grid behind data —————
ax.set_axisbelow(True)

# — 7. Data source annotation —————
fig.text(0.01, -0.02, 'Source: Dataset Name | Author: Your Name',
        ha='left', fontsize=8, color='gray')

# — 8. Save —————
plt.tight_layout()
plt.savefig('plot.png', dpi=300, bbox_inches='tight') # high-res
plt.show()
```

20.2 Colors & Colormaps

Color Purpose	Recommended Choice	Example Code
Single series	Named color or hex	color='#1565C0'
Categorical groups	qualitative palette	palette='Set2' or 'tab10'
Sequential data (count, density)	Blues, YlOrRd, viridis	cmap='Blues'
Diverging data (correlation, change)	coolwarm, RdBu, PiYG	cmap='coolwarm'
Colorblind-safe	colorblind palette	sns.set_palette('colorblind')

20.3 Saving Plots

Saving Plots in Different Formats

```
# High-resolution for publication/print
plt.savefig('figure.png', dpi=300, bbox_inches='tight', facecolor='white')

# PDF for vector graphics (best for papers/reports)
plt.savefig('figure.pdf', bbox_inches='tight')

# SVG for web / editing in Illustrator
plt.savefig('figure.svg', bbox_inches='tight')

# Transparent background
plt.savefig('figure.png', dpi=300, transparent=True, bbox_inches='tight')
```


21. Quick Reference — Choose the Right Plot

Use this table as a cheat sheet. Find your question/data type in the first column, then pick the chart.

Your Data / Question	Best Chart	Alternative
Compare values across categories	Bar Chart (vertical)	Dot plot, Lollipop chart
Compare distributions across groups	Box Plot	Violin Plot, Strip Plot
Show distribution of ONE numeric var	Histogram + KDE	ECDF, Violin Plot
Show relationship: two numeric vars	Scatter Plot	Hexbin (large data)
Show trend over time	Line Chart	Area Chart
Show part-to-whole (≤ 5 categories)	Pie / Donut Chart	Stacked Bar Chart
Show part-to-whole OVER TIME	Stacked Area Chart	100% Stacked Bar
Correlation matrix / grid data	Heatmap	Pair Plot
Classify model evaluation	Confusion Matrix Heatmap	ROC Curve
Regression model performance	Scatter: Predicted vs Actual	Residual Plot
Feature importance	Horizontal Bar Chart (sorted)	SHAP beeswarm
High-dimensional data in 2D	t-SNE / UMAP Scatter	PCA Biplot
All pairwise relationships	Pair Plot (sns.pairplot)	Correlation Heatmap
Count per category	Count Plot / Bar Chart	Treemap
Large scatter (> 5000 pts)	Hexbin Plot	Contour / density plot
Binary classifier performance	ROC Curve + AUC	Precision-Recall Curve
Group proportions	Grouped Bar or 100% Stacked Bar	Small multiples
Many categories (> 15)	Horizontal Bar (sorted, top-N)	Treemap
Ranked comparison	Horizontal Bar (sorted)	Lollipop chart
Interactive exploration	Plotly (any chart type)	Bokeh, Altair

Golden Rules of Data Visualization

- ✓1. TITLE: Every chart must have a specific, descriptive title. Never leave it blank.
- ✓2. AXES: Always label both axes with the variable name AND units.
- ⚠3. START AT ZERO: Bar charts MUST start at zero. Starting higher is a visual lie.
- ✓4. COLOR: Use colorblind-safe palettes. Avoid rainbow colormaps for continuous data.
- ✓5. LEGEND: Only add a legend when there are multiple groups/series.
- ⚠6. CLUTTER: Remove chart junk — 3D effects, excessive borders, gradients. Less is more.
- 📌7. AUDIENCE: Simplify for non-technical audiences; add detail for analysts.
- ✓8. RIGHT CHART: Match the chart type to your data type and the question being answered.